



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Satellite Data Visualization Platform for Weather and Climate Pattern Analysis

A CASE STUDY ANALYSIS REPORT

Submitted in the partial fulfilment for the Course of

CSA1012-Software Engineering

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

Artificial Intelligence & Machine Learning

Submitted by

Y.Navya(192425177)

Under the Supervision of

Dr.K.Anita Davamani



SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

AUGUST-2026



Table Of Contents

S.No	Table Of Contents	Page No.
1.	Analyze System Requirements	3-4
2.	Select a Suitable Software Architecture	4-5
3.	Software Architecture Diagram	5
4.	Components and Their Interactions	6-8
5.	Quality Attributes	8
6.	Justification of Architectural Decisions	9

Table Of images:

S.No	Name of the image	Page No.
1.	Fig.1.Software Architecture Diagram of the Satellite Data Visualization Platform	5

1. Analyze System Requirements

1.1 System Objectives

The proposed system aims to provide an easy-to-use web platform that enables users to visualize and analyze satellite-based weather and climate data. The primary objectives are:

- Develop a data ingestion pipeline to collect satellite imagery from publicly available sources such as NASA and NOAA.
- Display real-time and historical weather information through interactive maps.
- Analyze climate trends such as temperature, rainfall, cloud cover, and vegetation over time.
- Detect weather anomalies including cyclones, droughts, and unusual temperature changes.
- Allow users to compare multiple satellite data layers for better analysis.
- Enable researchers to export processed data and visualizations.
- Provide region-specific filtering for localized weather analysis.
- Offer a simple and user-friendly web interface for both technical and non-technical users.

1.2 Functional Requirements

The system should provide the following functions:

- User registration and secure login.
- Authentication and role-based authorization.
- Retrieve satellite data from NASA and NOAA APIs.
- Store raw satellite images and processed climate data.
- Process and clean raw satellite datasets.
- Generate interactive maps, graphs, and heatmaps.
- Detect weather anomalies using pattern detection algorithms.
- Filter climate information by region and date.
- Compare multiple weather datasets.
- Export reports, graphs, and processed datasets.
- Provide administrative functions for managing users and system settings.

1.3 Non-Functional Requirements

The system must satisfy the following non-functional requirements:

- **Performance:** Fast processing and visualization of large satellite datasets.
- **Scalability:** Ability to handle increasing data volume and user requests.
- **Reliability:** Continuous and accurate processing with minimal failures.
- **Availability:** The platform should be accessible 24×7.
- **Security:** Secure authentication, encrypted communication, and protected APIs.
- **Maintainability:** Modular architecture for easy updates and maintenance.
- **Usability:** Simple and responsive web interface suitable for non-expert users.

1.4 Key Quality Attributes

Quality Attribute	Description
Scalability	Supports growing satellite datasets and increasing users.
Maintainability	Modular components allow easy updates and enhancements.
Reliability	Ensures accurate data processing and stable system operation.
Availability	Provides continuous access to users with minimal downtime.
Performance	Delivers fast visualization and processing of satellite data.
Security	Protects user information and external API communication.

2. Select a Suitable Software Architecture

Selected Architecture: Hybrid Architecture (Layered + Microservices)

The proposed system uses a Hybrid Architecture, combining Layered Architecture with Microservices.

Architecture Layers

- **Presentation Layer:** Web Interface
- **Application Layer:** API Gateway
- **Business Layer:** Authentication, Data Ingestion, Data Processing, Visualization,

Pattern Detection, and Export Services

- **Data Layer:** PostgreSQL Database and Satellite Image Storage
- **External Layer:** NASA API and NOAA API

Justification

The Hybrid Architecture is the most suitable because:

- Each service performs an independent task, making the system modular.
- Microservices allow independent development, deployment, and scaling.
- The layered structure separates user interface, business logic, and data management.
- New satellite APIs and analysis modules can be integrated without affecting existing components.
- The architecture supports cloud deployment and future expansion.
- Faults in one service do not affect the entire system, improving reliability.

3. Software Architecture Diagram

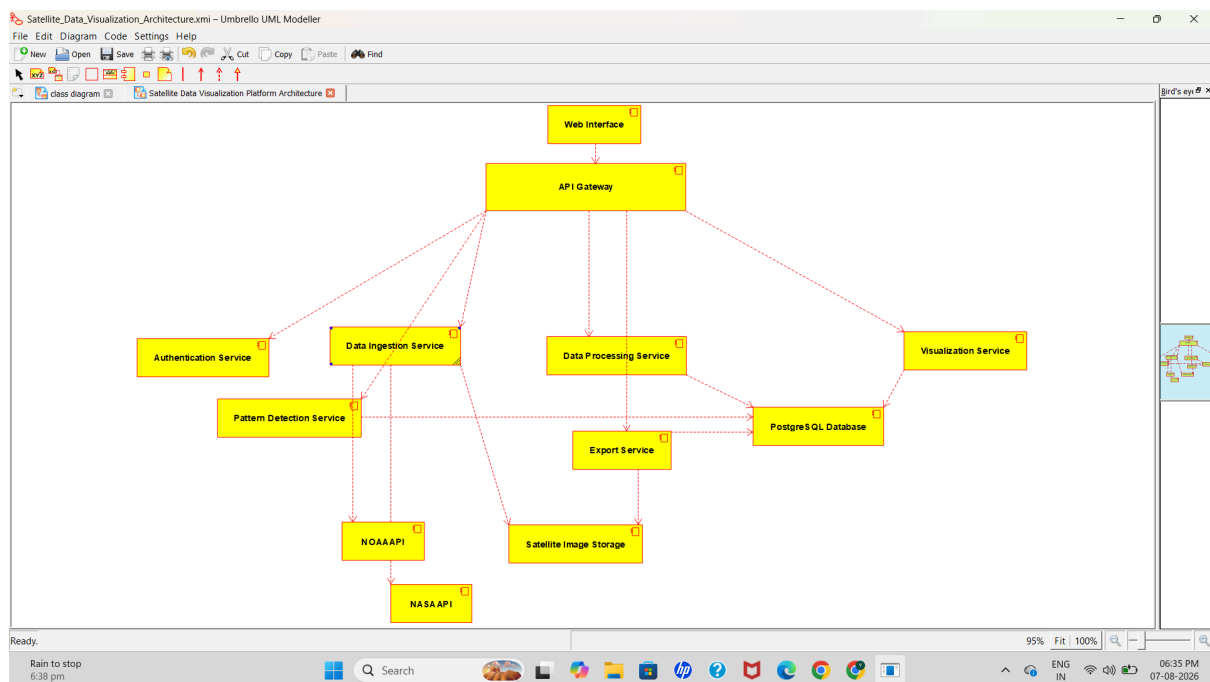


Figure 1: Software Architecture Diagram of the Satellite Data Visualization Platform

4. Explain Components and Their Interactions

4.1 Components

Web Interface

Provides a user-friendly interface where users can log in, view satellite maps, apply filters, analyze weather trends, and download reports.

API Gateway

Acts as the entry point for all client requests. It validates requests, handles routing, and forwards them to the appropriate service.

Authentication Service

Manages user registration, login, authentication, and role-based authorization to ensure secure access.

Data Ingestion Service

Retrieves real-time and historical satellite data from NASA and NOAA APIs and stores the raw data in Satellite Image Storage.

Data Processing Service

Processes raw satellite data, removes errors, converts it into usable formats, and generates climate-related information.

Visualization Service

Creates interactive maps, graphs, charts, and heatmaps from the processed data for easy analysis.

Pattern Detection Service

Analyzes processed satellite data to identify weather anomalies such as cyclones, droughts, floods, and temperature spikes.

Export Service

Allows users to export processed datasets, reports, graphs, and satellite images for further research.

PostgreSQL Database

Stores processed climate information, user details, metadata, and system logs.

Satellite Image Storage

Stores large satellite images and historical datasets.

NASA API

Provides satellite imagery and climate-related datasets.

NOAA API

Supplies weather observations and satellite-based environmental information.

4.2 Component Interactions

- The user accesses the platform through the **Web Interface**.
- The **API Gateway** receives all user requests.
- The **Authentication Service** verifies user credentials.
- The **Data Ingestion Service** retrieves satellite data from NASA and NOAA APIs.
- The **Data Processing Service** cleans and processes the retrieved data.
- The **Pattern Detection Service** analyzes processed data to detect weather anomalies.
- The **Visualization Service** generates maps, charts, and graphical reports.
- The **Export Service** prepares downloadable reports and datasets.
- The **PostgreSQL Database** stores processed information, while **Satellite Image Storage** stores large satellite images.

4.3 Overall Workflow

1. User logs into the system.
2. The API Gateway validates and routes the request.

3. Satellite data is collected from NASA and NOAA APIs.
4. Raw satellite images are stored in Satellite Image Storage.
5. Data Processing Service converts raw data into meaningful climate information.
6. Pattern Detection Service identifies significant weather events.
7. Visualization Service displays interactive maps and graphs.
8. Users analyze the results and export reports if required.

5. Discuss Quality Attributes

Scalability

The architecture uses independent services that can be scaled individually. Additional servers can be added to handle increased users or larger satellite datasets.

Security

The Authentication Service provides secure login and role-based access control. HTTPS encryption protects communication, and APIs are securely integrated.

Performance

Separate processing and visualization services improve performance. Frequently accessed data can be cached to reduce response time.

Reliability

Microservices isolate failures, ensuring that problems in one service do not stop the entire system. Backup storage and database recovery further improve reliability.

Maintainability

The layered and modular architecture allows developers to update or modify one service without affecting other components, making maintenance easier.

Availability

The system can be deployed in the cloud with load balancing and backup servers, ensuring continuous service availability and high uptime.

6. Justify Architectural Decisions

Rationale

A Hybrid Architecture (Layered + Microservices) was selected because it provides both modularity and scalability. The layered structure separates presentation, business logic, and data management, improving organization and maintainability. Microservices enable each major function, such as data ingestion, visualization, and pattern detection, to operate independently. This makes the system more reliable, easier to maintain, and capable of handling large volumes of satellite data efficiently.

Trade-offs

Advantages

- Modular and organized design.
- High scalability for future growth.
- Easy integration of additional satellite data sources.
- Improved fault isolation.
- Simplified maintenance and testing.
- Supports cloud deployment and distributed processing.

Disadvantages

- More complex deployment compared to a monolithic architecture.
- Increased communication between services.
- Higher infrastructure and maintenance costs.

Support for Future Enhancements

The proposed architecture provides a strong foundation for future development. Additional satellite data providers, machine learning models for weather prediction, advanced visualization techniques, and mobile applications can be integrated without major architectural changes. The modular design also simplifies long-term maintenance and allows the system to evolve as new technologies and user requirements emerge.